Cybersteps CVE DB

Project Summary for Submission Item 4: quick run guide, design choices, and a real bug fix from development.

Project Overview

This project is a small CVE database system built with Python and SQLite. It supports manual entry, bulk import from CSV and JSON, live syncing from the National Vulnerability Database, SQL-based analysis, and an AI feature that turns natural-language questions into safe SQL queries.

Quick Guide: How To Run The Tool

1. Open a terminal in the project folder.
2. Create a virtual environment: `python -m venv venv`
3. Activate it on Windows: `.\venv\Scripts\activate`
4. Install packages: `pip install -r requirements.txt`
5. Create a `.env` file if you want NVD sync or AI features.
6. Run the CLI: `python main.py`
7. Optional: run the web UI: `streamlit run ui.py`

```
CVE_DB_PATH=cve.db
NIST_API_KEY=your_nist_key_here
CEREBRAS_API_KEY=your_cerebras_key_here
GEMINI_API_KEY=your_gemini_key_here
```

Why I Chose Cerebras

I opted for Cerebras mainly because speed matters a lot for the AI query feature. When a user asks a natural-language question and expects SQL plus results back quickly, low latency makes the tool feel much more interactive and practical.

Cerebras advertises inference speeds of up to 3,000 tokens per second, which made it a strong fit for this project because it helps reduce waiting time between a question, the generated SQL, and the final database results.

Why I Chose This Structure

Relational schema

I used four tables: `cves`, `vendors`, `products`, and `cve_products`. That keeps the main vulnerability record separate from the affected software relationships and makes SQL analysis much easier.

I chose this schema because it stays simple enough for the assignment while still supporting realistic joins, vendor/product analysis, and many-to-many relationships between CVEs and affected software.

Separated Python modules

I split the program into focused files instead of putting everything in one script. For example, database helpers live in `db.py`, imports in `imports.py`, sync in `sync.py`, and AI logic in `query.py`.

This structure makes the project easier to explain, test, and extend. It also mirrors how real projects are usually organized: shared infrastructure in one place and user-facing features in separate modules.

The Most Annoying Bug I Had To Fix

The trickiest bug happened in the AI query feature. One Cerebras model rejected the `reasoning_effort` parameter with an HTTP 422 error, while Gemini sometimes returned truncated JSON that failed validation before the SQL could run.

The fix was to make the AI planner more defensive:

- retry Cerebras requests without model-specific reasoning parameters if the model does not support them
- increase Gemini output length
- allow multiple planning attempts when the JSON or SQL does not validate on the first try

That bug taught me an important lesson: AI features are not just about prompting. You also need validation, retries, and safe fallbacks so the whole tool stays reliable.

Coolest Thing I Learned

The most interesting part of the project was connecting natural language to a real SQL database safely. It was not enough to generate SQL. I had to restrict the model to known tables, block unsafe statements, validate the query with SQLite, and sometimes ask the user a clarification question before running anything.

One of the last improvements was prompt-engineering the planner so it asks better follow-up questions when a request is materially ambiguous. For example, instead of guessing what words like `recent`, `office`, or `most vulnerable` mean, I adjusted the system prompt so the model asks for a time range, a target product, or a ranking metric first.

That made the feature feel much more practical and realistic than a simple chatbot answer.